Sprint 3 – Individual Retrospective Kamogelo Mosehle

Software Design - 2026

What I Built

My contributions this sprint were centred on the student dashboard and payment integration. I added a **Profile Settings tab** to the student dashboard, giving students a dedicated view of their account details alongside a sign-out button, previously there was no way to log out from within the dashboard. I also implemented the **My Sales** and **My Purchases tabs**, allowing sellers to see items they have sold and book drop-off slots at the campus trade facility, and allowing buyers to track their purchases, collection slots, and any outstanding cash shortfall amounts.

On the payment side, I built the full PayFast integration, the initiation flow, the sandbox confirmation endpoint, the slot booking UI on the success page, and the cash shortfall option that lets buyers split their payment between online and cash at the facility. I chose PayFast over Peach Payments because its sandbox environment is significantly easier to configure and test without a business account.

For testing and coverage, I wrote Jest tests for the payment integration on both the client and server, and successfully configured Codecov so that coverage reports are now visible on the Codecov dashboard (something that was not working in Sprint 2).

What I Found Difficult

The most significant challenge this sprint was the environment variable mismatch between local development and the deployed site. The client was reading `VITE_API_URL` in payment-related components while listings used a separate `API_BASE_URL` config — both were supposed to point to the backend, but only one was correctly set in the GitHub Actions build pipeline. This meant the deployed frontend sent payment requests to `http://localhost:8080` instead of the Azure App Services URL, causing all payment, sales, and purchases functionality to fail with a "Failed to fetch" error on the live site.

Catching this required reading the browser network tab carefully and tracing the request URL back to the missing environment variable.

A secondary difficulty was a double-decrement bug on facility slot counts, caused by a DB trigger firing at the same time as a manual RPC call. Diagnosing it required checking both the backend code and the Supabase database trigger configuration simultaneously.

What I Would Do Differently

I would establish a deployment test checklist before any assessment deadline, a short list of critical user flows to verify on the deployed URL, not just locally. Had I tested the Buy Now button on the Azure-hosted site even once before submission, the environment variable bug would have been caught immediately. I would also consolidate all backend URL references into a single client-side config from the start of the sprint, rather than having two separate constants that can diverge silently.

How My Work Fits the Overall System

The payment integration, sales tracking, and profile management features I built this sprint complete the core transaction lifecycle of the marketplace. Without these components, the system was a browsing tool; students could see listings but had no mechanism to buy, track, or manage items. My work connects the frontend listing experience built by other team members directly to the PayFast gateway, the Supabase database, and the campus trade facility slot system, making the platform functional end-to-end for both buyers and sellers.

Reflection

This sprint exposed how quickly complexity compounds as a system matures. Adding payment meant handling edge cases, partial payments, slot ordering between buyer and seller, RLS policy creation on Supabase, and environment-specific configuration, that simply did not exist in earlier sprints. The deployed site not working during the client assessment is disappointing and entirely avoidable with better pre-submission testing discipline. That is the clearest lesson I am taking into Sprint 4, and I'll try to do better.